

MIA Benchmark + MUSE 项目 —— 计算机保研复试问答手册

论文信息：本文档基于对 mia-benchmark 项目代码（约 3000+ 行 Python）以及同名论文终稿（44 页，含 5 张图、4 张表、37 篇参考文献）的深度研读，结合计算机保研复试（参考 CS-BAOYAN 社区）常见面试风格，整理而成。

论文核心贡献：

- 构建了面向样本级机器遗忘的**成员推理攻击统一评估框架**，覆盖 3 个数据集 × 4 种遗忘方法 × 4 种攻击方法，从 5 个维度进行系统评估
- 提出了面向成员推理攻击防御的遗忘方法 **MUSE**（Membership-Undistinguishable Sample Erasure），通过分布对齐和差异压缩降低隐私泄露风险

目录

- 项目概览与背景知识
- 核心概念深度解析
- 论文框架与评估体系
- MUSE 方法技术细节
- 四种攻击方法对比
- 实验设计与结果分析
- 工程实现与架构
- 多轮实验与可复现性
- 论文研究内容与未来方向
- 面试综合策略

一、项目概览与背景知识

Q1：请用一句话介绍这个项目。

参考答案：这是一个面向 **sample-wise machine unlearning**（样本级机器遗忘）的统一基准测试平台（benchmark），它集成了 4 种主流成员推理攻击（MIA）方法和 5 种以上的机器遗忘方法，并提出了一种新的遗忘方法 **MUSE**（Membership-Undistinguishable Sample Erasure）。

应答技巧：

- 用“是什么 + 做什么 + 创新点”三句话概括
- 提及 AI 安全 / 隐私保护这个热门方向，引发面试官兴趣
- 关键词：machine unlearning、membership inference attack、benchmark、隐私合规

Q2：什么是机器遗忘（Machine Unlearning）？为什么需要它？

参考答案：

定义：机器遗忘是指从已训练好的模型中“删除”特定训练样本的影响，使模型的行为仿佛从未见过这些数据。

法律驱动力：

- 欧盟 **GDPR**（通用数据保护条例）第 17 条规定了“被遗忘权”（Right to be Forgotten）
- 美国 **CCPA**（加州消费者隐私法案）也有类似规定
- 中国《**个人信息保护法**》第 47 条规定个人有权请求删除其个人信息

技术挑战：

- 深度学习模型是高度非线性的黑盒系统，无法像删除数据库记录那样简单地“删掉”一条数据
- 直接重新训练（retrain）是最朴素的方法，但计算成本极高（尤其对于大模型）
- 需要在 **遗忘效果**（unlearning efficacy）和 **模型效用**（model utility）之间取得平衡

现实场景：

- 用户要求从人脸识别系统中删除自己的面部数据
- 医疗 AI 模型中移除患者敏感信息
- LLM 中遗忘侵权训练数据

应答技巧：从"法律→技术→本项目定位"的逻辑链展开，展示你对领域 motivation 的理解。

Q3：什么是成员推理攻击（Membership Inference Attack, MIA）？

参考答案：

定义：MIA 是一种隐私攻击方法，攻击者通过分析模型的输出（如置信度向量、logits 等），判断某个特定数据样本是否被用于训练该模型。

核心原理：

- 机器学习模型通常对训练过的样本（成员，member）表现出更高的置信度
- 对未见过的样本（非成员，non-member）则相对不确定
- 攻击者利用这种统计差异来推断成员身份

攻击场景举例：

- 判断某人的医疗记录是否被用于训练疾病预测模型 → 泄露该人患有某种疾病
- 判断某张人脸照片是否在训练集中 → 泄露该人的身份信息

在 Machine Unlearning 中的角色：

- MIA 是**评估遗忘效果的核心工具**：如果遗忘成功，那么被遗忘的样本在 MIA 测试中应该显示为"非成员"
- 一个理想的遗忘方法应该让 MIA 的 AUC 接近 0.5（随机猜测水平）

应答技巧：用"角色类比"帮助理解——MIA 就像一个侦探，通过观察模型的"反应"来判断某个人是否曾经"见过"某个样本。

Q4：本项目的核心贡献是什么？

参考答案：

1. **统一的 Benchmark 框架：**
 - 单一入口 `run_benchmark.py`，统一调度 `pretrain` → `shadow` → `unlearn` → `reminiscence` → `attack` 五个阶段
 - 通过 `benchmark/samplewise.py` 实现数据集预设、阶段编排和攻击分发的全流程自动化
2. **多攻击方法的公平对比：**
 - 接入 LiRA、REA、RULI、UnlearningLeaks 四种主流 MIA 方法
 - 所有攻击共享同一套 `forget index`、`shadow models` 和随机种子，保证对比的公平性
3. **多遗忘方法 Baseline：**
 - 提供 `retrain`（理论上界）、`finetune`、`negative_grad`、`scrub`、`FisherForgetting`、`salun` 等 10+ 种方法
 - 每种方法在多个数据集上各有预设超参数
4. **新方法 MUSE：**
 - 基于"让遗忘样本在模型输出上看起来像非成员样本"的核心思想
 - 通过分布对齐损失（`L_align`）和统计量对齐损失（`L_stat`）实现更优的遗忘效果
5. **多轮可复现实验：**
 - 提供 shell 脚本支持多轮实验（如 `scripts/run_cifar10_five_rounds_muse.sh`）
 - 自动计算 `mean/std`，保证实验结论的统计显著性

二、核心概念深度解析

Q5：sample-wise unlearning 和 class-wise unlearning 有什么区别？

参考答案：

维度	Sample-wise	Class-wise
遗忘粒度	单个/一组样本	整个类别
难度	更细粒度、更困难	相对粗粒度
评价方式	需要 MIA 判定单个样本是否被遗忘	可以通过该类别准确率下降来评估
实际场景	用户要求删除自己的特定数据	从不安全的数据集中删除某个类别
本项目对应	本项目采用 sample-wise（随机选择 10% 训练样本为 forget set）	—

代码体现： forget_sample_main.py:172-175

```
forget_set_idx = np.random.choice(
    np.arange(len(trainset)),
    size=int(len(trainset) * args.forget_perc),
    replace=False
)
```

应答技巧：先对比两个概念，再说明本项目选择 sample-wise 的原因——更贴近实际应用场景（"删除特定用户的数据"），技术上也更具挑战性。

Q6：解释项目中 TA、UA、RA、ZRF、MIA 这些指标的含义。

参考答案：

指标	全称	计算方式	含义
TA (Test Accuracy)	测试准确率	模型在测试集上的分类准确率	衡量模型的 通用性能 （越高越好）
UA (Unlearning/Forgetting Accuracy)	遗忘集准确率	模型在 forget set 上的准确率	衡量遗忘 是否彻底 （越低越好，理想值 = 随机猜测）
RA (Remaining/Retain Accuracy)	保留集准确率	模型在 retain set 上的准确率	衡量遗忘的 精确性 （越高越好，不应影响保留样本）
ZRF (Zero-Retrain Forgetting)	零重训遗忘分数	1 - JSD(遗忘后模型，参考模型)	衡量遗忘模型的输出分布与从未见过遗忘数据的参考模型的接近程（越高越好）
MIA	成员推理攻击概率	用逻辑回归分类器判断遗忘样本	遗忘样本被识别为"成员"的比例（越低越好）
AUC	ROC 曲线下面积	MIA 分类器的 AUC 值	衡量攻击者区分成员/非成员的能力（越接近 0.5 越好）
TPR@FPR	特定假阳率下的真阳率	固定 FPR 阈值下的 TPR	严格场景下的隐私风险指标

代码体现： forget_random_strategies.py:66-76

```
def get_metric_scores(...):
    d_t_acc_dict = evaluate(model, valid_dataloader, device) # TA (测试集)
    d_f_acc_dict = evaluate(model, forget_train_dataloader, device) # UA (遗忘集)
    d_r_acc_dict = evaluate(model, retain_train_dataloader, device) # RA (保留集)
    zrf = UnLearningScore(model, unlearning_teacher, forget_train_dataloader, ...) # ZRF
    mia = get_membership_attack_prob(...) # MIA
```

应答技巧：解释每个指标时，同时说明"理想值"应该是什么，这样能展示你对评估体系的理解深度。

Q7: ZRF (Zero-Retrain Forgetting) 是如何计算的？为什么重要？

参考答案：

计算方式（utils/metrics.py:20-34）：

```
def UnLearningScore(tmodel, gold_model, forget_dl, batch_size, device):
    # tmodel: 遗忘后的模型
    # gold_model: 原始预训练模型（作为参考/教师模型）
    # 计算两个模型在 forget set 上的 softmax 分布的 JS 散度
    # ZRF = 1 - JSDiv(model_preds, gold_model_preds)
```

- tmodel（遗忘后模型）和 gold_model（原始模型，作为 unlearning_teacher）分别在 forget set 上做推理
- 收集两者的 softmax 概率分布
- 计算 JS 散度（Jensen-Shannon Divergence）
- **ZRF = 1 - JSD**，值越大说明遗忘后模型与原始模型的输出分布差异越大

为什么重要：

- ZRF 是一种与 **MIA 独立** 的遗忘质量评估指标，不需要额外的影子模型或攻击训练
- 它直接衡量“遗忘前后模型行为变化的程度”
- 但它有局限性：ZRF 高不一定代表隐私保护好（可能模型只是整体性能退化了）

应答技巧：强调 ZRF 是“内部指标”，MIA 是“外部攻击验证指标”，两者互补。

Q8: shadow model 是什么？为什么 MIA 需要它？

参考答案：

定义：Shadow model（影子模型）是在与目标模型**相同的数据分布**上、使用**不同的随机子集**训练出来的辅助模型。它们模拟目标模型的行为，但攻击者知道它们的训练集成员身份（ground truth）。

为什么需要：

- 攻击者不知道目标模型的训练集构成，无法直接训练 MIA 分类器
- Shadow models 提供**已知标签的（特征，成员/非成员）**训练数据
- 利用 shadow models 训练的分类器可以**迁移**到目标模型上

本项目的实现（train.py:160-168）：

```
if args.num_shadow is not None:
    # 每个 shadow model 使用不同的随机子集（pkeep=0.83 表示保留 83% 数据）
    keep = np.random.uniform(0, 1, size=(args.num_shadow, dataset_size))
    order = keep.argsort(0)
    keep = order < int(args.pkeep * args.num_shadow)
    keep = np.array(keep[args.shadow_id], dtype=bool)
```

- 每个 shadow model 使用约 83% 的数据训练
- 每张图片约有一半的 shadow models “见过”它（成员），另一半“没见过”（非成员）
- 共训练 8 个 shadow models（--num-shadow 8）

面试加分点：

- Shadow model 的思想源自 [Shokri et al., 2017](#) 的奠基性工作
- 近年来也有 shadow-model-free 的 MIA 方法（如用数据增强模拟、基于损失阈值等），但在 unlearning 场景下 shadow-based 方法仍是主流

三、论文框架与评估体系

Q9：论文的整体结构是怎样的？各章节如何组织？

参考答案：

论文共 5 章，采用“理论 → 攻击评估框架 → 防御方法 → 结论”的递进式结构：

章节	标题	核心内容
第 1 章	绪论 (Introduction)	研究背景与意义（隐私法规、机器遗忘需求）、国内外研究现状（遗忘方法/MIA/两者交叉）、研究内容、论文组织
第 2 章	相关理论基础 (Theoretical Background)	机器遗忘的形式化定义与分类、MIA 原理与评估指标、面向攻击防御的遗忘方法概述
第 3 章	面向机器遗忘的 MIA 评估框架	核心贡献一： 框架设计（4 组件）、5 维评估体系、实验协议、跨数据集/跨方法/跨攻击实验分析
第 4 章	面向 MIA 防御的遗忘方法 MUSE	核心贡献二： MUSE 设计动机、三损失函数、算法流程（11 步伪代码）、对比实验与消融分析
第 5 章	总结与展望 (Conclusion)	研究总结、5 个未来方向（更大数据集、差分隐私结合、自适应攻击、MUSE 改进、良性数据威胁）

论文的核心论证逻辑：

- 1. 先证明"现有遗忘方法的评估不够充分"（第 3 章用 5 个维度全面评估）
- 2. 再提出"面对 MIA 攻击，需要专门设计的防御式遗忘方法"（第 4 章 MUSE）

Q10：论文提出的 MIA 评估框架包含哪 5 个评估维度？为什么需要多维评估？

参考答案：

论文在第 3 章提出从 **5 个维度**全面评估攻击方法的表现：

维度	英文对应	评估内容	核心指标
有效性	Effectiveness	攻击能否成功区分成员/非成员	AUC、Accuracy、TPR@FPR
稳定性	Stability	攻击在不同数据集/方法上的表现一致性	跨数据集的 Accuracy 方差
适用性	Applicability	攻击方法对不同遗忘方法的覆盖范围	可成功运行的 (方法, 攻击) 组合数
成本	Cost	攻击所需的时间和计算资源	运行时间（秒）、Shadow 模型数
评估真实性	Evaluation Authenticity	评估协议与实际威胁模型的吻合度	是否共享 forget index、shadow 是否独立训练

为什么需要多维评估：

- 1. **单一指标的局限性：**
 - AUC 高但 TPR@低FPR 低 → 攻击在严格场景下无效
 - Accuracy 高但只在某些遗忘方法上有效 → 适用性差
 - 效果好但需要 100 个 shadow models → 攻击成本过高，不现实
- 2. **实验设计中的发现**（论文 3.4 节）：
 - RULI 在 NegGrad 上 Accuracy 最高（CIFAR-100 上达 70.41%），但运行时间最长（~7000s）
 - LiRA 和 ReA 稳定性最好（跨数据集方差小），但需要 reminiscence 阶段
 - Unleaks 适用性最广但 Accuracy 偏低
- 3. **实际意义：** 多维评估帮助研究者理解"哪种攻击在什么条件下对哪种遗忘方法最有效"，从而做出更有针对性的防御设计。

面试应答技巧： 强调"多维评估"的设计思路反映了你对"公平对比"的深入思考——不是简单的跑实验比数字，而是建立一个评估方法论（evaluation methodology）。

Q11：论文的实验协议（Experimental Protocol）如何保证评估的公平性？

参考答案：

论文 3.3.2 节给出了完整的实验协议（对应 Figure 3-1 的 Benchmark Pipeline）：

- 1. 统一的随机采样：
 - 随机选取 10% 训练样本作为 forget set
 - 固定随机种子，所有方法共享同一份 forget index（存储在 random_index_set/ 目录下）
- 2. 统一的模型初始化：
 - 所有遗忘方法从**同一个预训练模型**出发（pretrain 阶段）
 - 使用相同的训练 epoch 和数据增强策略
- 3. 统一的 Shadow 模型复用：
 - 预训练 8 个 shadow models，所有攻击方法复用
 - 避免因不同 shadow 质量导致的评估偏差
- 4. 多轮实验取均值：
 - 每个实验跑 5 轮（论文数据中带 ± 标准差）
 - 每轮更换随机种子，保证统计显著性
- 5. 标准化的输出指标：
 - 统一使用 AUC、Accuracy、TPR@0.1%FPR、TPR@1%FPR、TPR@10%FPR
 - 通过 AttackResult 数据类统一封装

四、MUSE 方法技术细节

Q12：请详细介绍 MUSE 方法的核心思想。

参考答案：

MUSE（Membership-Undistinguishable Sample Erasure）的核心思想是：“让被遗忘样本在模型输出统计特征上变得与非成员（从未见过的）样本难以区分”。

直觉理解：

- 一个“见过”某张图片的模型，通常会对该图片的分类更“自信”（高置信度）
- MUSE 的目标是消除这种过度自信，使得遗忘样本的输出统计量与从未训练过的参考样本相匹配

三个损失函数：

$$L_{total} = L_{base} + \lambda_{align} \times L_{align} + \lambda_{stat} \times L_{stat}$$

损失项	计算对象	目标
L_base	保留集 (D_r)	CrossEntropy(model(x_r), y_r) — 保持对保留数据的分类能力
L_align	遗忘集 vs 参考集	MSE(batch_mean_softmax(D_f), batch_mean_softmax(D_t)) — 全局分布对齐
L_stat	遗忘集 vs 参考集	MSE(stat(D_f), stat(D_t)) — 攻击敏感统计量对齐
		stat 包含：max confidence + top1-top2 margin

每步采样：

- 遗忘batch D_f → 需要被遗忘的样本
- 保留batch D_r → 需要保持性能的样本
- 参考batch D_t → 从未训练的测试集样本（作为“非成员”的参考）

代码实现： muse/trainer.py:59-116

```
def train_muse_epoch(model, retain_loader, forget_loader, reference_loader, optimizer, cfg):
    forget_iter = cycle(forget_loader)    # 循环迭代，防止长度不匹配
    reference_iter = cycle(reference_loader)
    for retain_batch in retain_loader:
        forget_batch = next(forget_iter)
        reference_batch = next(reference_iter)
        # 三步推理
        retain_logits = model(x_r)
        forget_logits = model(x_f)
        reference_logits = model(x_t)
        # 计算三个损失
        loss_base = CE(retain_logits, y_r)
        loss_align = compute_alignment_loss(forget_logits, reference_logits)
        loss_stat = compute_stat_loss(forget_logits, reference_logits)
        loss_total = loss_base +  $\lambda_{align}$ *loss_align +  $\lambda_{stat}$ *loss_stat
```

应答技巧：用一个生动的类比——"MUSE 就像一个魔术师，让曾经见过的人（遗忘样本）在模型面前装作陌生人（非成员样本），同时还要保持对其他熟人（保留样本）的正常辨认能力。"

Q13：为什么 MUSE 选择对齐 max confidence 和 top1-top2 margin 而不是其他统计量？

参考答案：

这两个统计量是 MIA 攻击者最常用的**判别信号**：

- 1. Max Softmax Confidence（最大 softmax 置信度）：**
 - 训练过的样本通常有更高的预测置信度
 - 是 LiRA score 计算的重要输入（通过 logit rescaling）
 - 直接对齐这个量可以"欺骗"基于置信度的攻击
- 2. Top1-Top2 Logit Margin（最大和第二大 logit 的差值）：**
 - 反映了模型对预测结果的"确定程度"
 - 成员样本的 margin 通常更大
 - 对齐 margin 使得模型对遗忘样本的预测变得"犹豫"

理论依据（ muse/trainer.py:36-41 ）：

```
def _compute_batch_stats(logits):
    probs = F.softmax(logits, dim=1)
    max_confidence = probs.max(dim=1).values
    top2_logits = torch.topk(logits, k=2, dim=1).values
    margins = top2_logits[:, 0] - top2_logits[:, 1]
    return torch.stack([max_confidence.mean(), margins.mean()])
```

为什么取 batch-mean：

- 单个样本的统计量噪声较大，不够稳定
- batch-mean 作为分布的一阶矩估计，提供了一个平滑的优化目标
- 避免过度拟合单个样本

额外思路（面试加分）：

- 也可以考虑对齐 entropy、输出分布的方差等高阶统计量
- 当前设计遵循 Occam's razor：用最少的额外统计量达到最大效果

Q14：MUSE 中的两个超参数 λ_{align} 和 λ_{stat} 是如何设置的？有什么调参经验？

参考答案：

数据集默认值（ config.py:68-93 ）：

数据集	lr	epochs	λ_{align}	λ_{stat}
Cifar10	4e-4	7	1.0	0.5
Cifar100	3e-4	8	1.0	0.75
Cinic10	4e-4	8	1.0	0.5
TinyImageNet	3e-4	10	0.5	0.25

调参规律：

- 1. $\lambda_{align} > \lambda_{stat}$ 通常是合理的：分布对齐是基础目标，统计量对齐是精细调整
- 2. 大数据集/复杂任务减小 λ ：TinyImageNet（200 类）的 λ 明显小于 CIFAR10（10 类），因为更大的输出空间需要更谨慎的扰动
- 3. λ_{stat} 随类别数增加而减小：CIFAR10(0.5) $\rightarrow$ CIFAR100(0.75) 是一个例外，因为 CIFAR100 上 MUSE 的收敛更困难，需要更强的统计约束

实现细节 - 支持命令行覆盖（benchmark/samplewise.py）：

```
--methods muse:4e-4:7 # 覆盖 lr 和 epochs
```

Q15：MUSE 和 SCRUB 方法有什么区别？

参考答案：

维度	MUSE	SCRUB
核心策略	分布对齐 + 统计量对齐	知识蒸馏 + max/min 交替训练
参考信号	非成员样本的输出统计量	原始教师模型的输出分布
训练阶段	单一阶段，同时优化三个损失	两阶段：前 msteps 最大化 forget loss，之后最小化 retain loss
遗忘机制	让遗忘样本“模仿”非成员	让遗忘样本“偏离”老师（通过最大化 forget loss）
复杂度	需要额外参考数据加载器	需要维护教师模型和学生模型

SCRUB 的关键代码（forget_random_strategies.py:945-1084）：

```
if epoch <= args.msteps: # 前几个epoch: 最大化forget loss
    maximize_loss = train_distill(..., "maximize")
# 后几个epoch: 最小化retain loss
train_acc, train_loss = train_distill(..., "minimize")
```

本质区别：

- SCRUB 是 **teacher-student 范式**（用原始模型指导遗忘），目标是让遗忘样本的输出偏离原始决策边界
- MUSE 是 **distribution-matching 范式**（用非成员样本指导遗忘），目标是让遗忘样本在统计上不可区分

五、四种攻击方法对比

Q16：四种攻击方法（LiRA / REA / RULI / UnlearningLeaks）的核心区别是什么？

参考答案：

1. LiRA（Likelihood Ratio Attack）

- 出处：Carlini et al., 2022 (Google Brain)
- 核心思想：使用 shadow models 的 logit 分布拟合高斯分布，用似然比检验判断成员身份
- 在线模式：同时使用 IN 和 OUT shadow models 的统计量
- 离线模式：仅使用 OUT models（更接近真实攻击场景）

关键代码（lira_utils.py:320-390）：

```
def lira_online(shadow_scores, shadow_in_out_labels, target_scores, target_in_out_labels):
    # 计算每个 shadow model 上 IN/OUT 样本的 logit 均值和方差
    mean_in = np.median(dat_in, 1)
    std_in = np.std(dat_in, 1)
    # 对目标样本计算似然比
    pr_in = -scipy.stats.norm.logpdf(sc, mean_in, std_in + 1e-30)
    pr_out = -scipy.stats.norm.logpdf(sc, mean_out, std_out + 1e-30)
    score = pr_in - pr_out
```

2. REA（Reminiscence-Enhanced Attack）

- 与 LiRA 的区别：攻击对象不是原始遗忘模型，而是经过**额外 reminiscence 阶段**的模型
- **Reminiscence 阶段**：对遗忘后的模型再用遗忘数据做少量训练，使得原本被“擦除”的痕迹重新浮现
- **为什么需要**：某些遗忘方法可能只是“掩盖”而非“删除”了记忆，通过 reminiscence 可以重新激活这些残留信息
- **代码体现**：需要额外运行 rea_reminiscence_random.py，攻击阶段复用 mia_lira.py 但 checkpoint 来自 reminiscence 目录

3. RULI（Robust UNLearning Inspection）

- **出处**：来自 Ruli/ 第三方子模块
- **核心思想**：在遗忘前后的模型参数空间中分析差异，执行多种攻击变体（efficacy / privacy leakage / population attack / vulnerable sample attack）
- **特点**：
 - 维护自己独立的 shadow attack 资产（RULI 内部重新训练 shadow models）
 - 支持多种 task 模式：selective（样本级）、class-wise（类别级）、mixed、canary、privacy、vulnerable
 - 使用更复杂的攻击模型而非简单的似然比

本项目适配（attacks/ruli.py）：

- 自动映射 benchmark 的遗忘方法到 RULI 的命名（如 finetune→FT，muse→MUSE，negative_grad→NegGrad）
- 共享 forget index 文件保证一致性
- 自动判断是否需要重新训练 shadow

4. UnlearningLeaks

- **核心思想**：通过比较遗忘前后模型的输出变化来推断成员身份
- **特征构造**（6 种方式）：
 - direct_diff：直接相减 posterior_original - posterior_unlearned
 - sorted_diff：先排序再相减
 - direct_concat：直接拼接
 - sorted_concat：排序后拼接
 - l2_distance：L2 距离
 - basic_mia：仅使用原始后验
- **攻击分类器**：逻辑回归 / 决策树 / 随机森林 / MLP
- **特点**：为每个 shadow model 也执行遗忘操作，构造“遗忘前后”的配对特征

总结对比表：

维度	LiRA	REA	RULI	UnlearningLeaks
分析对象	Logits 分布	Reminiscence 后的 logits	模型参数 + 输出	遗忘前后输出变化
Shadow 依赖	强（8个共享）	强（与LiRA相同）	强（独立训练）	强（需对shadow也做unlearning）
复杂度	中	高（多一个阶段）	高	高
真实性	高	更高（考虑残留记忆）	高	高

应答技巧：先概括四种方法的共同点（都需要 shadow models、都基于统计差异），再逐一说明每种方法的独特之处。如果有面试官追问，可以深入讨论“哪种方法的威胁模型最现实”。

Q17：为什么 REA 需要额外的 reminiscence 阶段？这揭示了什么问题？

参考答案：

Reminiscence 的核心假设：某些遗忘方法可能并没有真正从模型参数中"删除"信息，而只是"压制"了模型对遗忘样本的响应。通过对遗忘后的模型再做一些额外的微调（reminiscence），这些被压制的记忆可能重新浮现。

操作流程（`rea_reminiscence_random.py`）：

- 1. 加载已遗忘的模型
- 2. 用遗忘集数据再做少量训练（微调）
- 3. 保存 reminiscence 后的模型
- 4. 在 attack 阶段攻击这个 reminiscence 后的模型

揭示的问题：

- 1. **"假遗忘"风险：**一些方法（如 finetune）可能只是让模型暂时"忘记"，记忆仍残留在参数中
- 2. **评估不够严格：**仅在遗忘后立即评估可能高估了方法的有效性
- 3. **安全的遗忘应该是不可逆的：**即使攻击者获得了遗忘后的模型并对其进行再训练，也无法恢复被遗忘的信息

面试加分点：这与 adversarial robustness 中的"自适应攻击"（adaptive attack）思想类似——评估防御时要假设攻击者知道并会尝试绕过防御机制。

Q18：UnlearningLeaks 的 6 种特征构造方式各有什么优劣？

参考答案：

特征构造方式	输入维度	优点	缺点
direct_diff	C（类别数）	最直观，直接反映遗忘引起的变化	可能丢失顺序信息
sorted_diff	C	对后验排序后再做差，消除类别顺序的影响	排序破坏了类别语义
direct_concat	2C	保留完整信息	维度更高，噪声更大
sorted_concat	2C	结合排序和拼接	维度高 + 丢失类别语义
l2_distance	1	最简洁的一维特征	信息损失最大
basic_mia	C	仅用原始模型后验，最基础	未利用遗忘信息

如何选择（`attacks/unlearningleaks.py:342-364`）：

- `direct_diff` 是**默认选择**：在信息量和复杂度之间取得平衡
- 对于类别数较多的数据集（如 Cifar100），建议用 `direct_diff` 或 `l2_distance` 控制维度
- `sorted_diff` 适合类别语义不重要、只关心"变化模式"的场景

六、评估体系与指标

Q19：项目是如何保证多攻击方法对比公平性的？

参考答案：

- 1. **统一的 forget index：**
 - 所有遗忘方法使用同一份 `forgetting_dataset_index_{forget_perc}.npy`
 - 存储在 `log_files/model/forget_random_main/{net}-{dataset}-{classes}/random_index_set/`
 - 首次运行自动生成并持久化，后续方法直接复用
- 2. **统一的随机种子：**
 - 每个实验轮次内，`pretrain`、`unlearn`、`attack` 共享同一个 `--seed`
 - 保证 shadow models 的采样也是一致的（`set_random_seed` in `lira_utils.py:145-151`）
- 3. **统一的 shadow models：**

- 需要 `--stages shadow` 阶段预训练 8 个 shadow models
- 所有攻击方法复用同一批 shadow checkpoints

4. 统一的评估指标：

- 输出格式标准化：AUC、Accuracy、TPR@0.1%FPR、TPR@1%FPR、TPR@10%FPR
- 通过 `AttackResult` `dataclass` 统一封装（`attacks/base.py:56-66`）

5. 多轮实验取均值：

- 提供 shell 脚本支持 5 轮实验，每轮换不同 seed
- 最终输出各指标的 `mean ± std`

代码体现（`benchmark/samplewise.py:622-665`）：

```
run_summary = {
    "dataset": cfg.dataset,
    "seed": args.seed,
    "forget_perc": args.forget_perc,
    "methods": [...],
    "attacks": [...],
    "num_shadow": args.num_shadow,
    "stage_runtime_sec": stage_runtime_sec,
}
```

Q20：TPR@10%FPR 为什么被认为是较严格的隐私指标？

参考答案：

ROC 曲线与 FPR-TPR 权衡：

- ROC 曲线绘制了所有可能阈值下的 TPR vs FPR
- AUC 是整体指标，可能高估“安全性”——因为高 FPR 区域的 TPR 实际上没有实际意义

为什么关注低 FPR 区域：

- 在实际攻击场景中，攻击者不能承受高假阳率（把太多非成员误判为成员）
- 一个攻击只有在低 FPR（如 0.1%、1%、10%）下取得高 TPR 才算真正有效
- 从隐私保护的角度，只要能抵御低 FPR 下的攻击，就可以认为保护有效

层次化的评估：

场景	指标	含义
最严格	TPR@0.1%FPR	几乎不允许误报的极端场景
严格	TPR@1%FPR	微小误报可接受
实用	TPR@10%FPR	本项目的推荐指标，平衡严格性和可测量性

代码实现（`lira_utils.py:393-400`）：

```
def cal_stats(final_preds, true_labels):
    fpr, tpr, thresholds = metrics.roc_curve(true_labels, final_preds, pos_label=1)
    auc = metrics.auc(fpr, tpr)
    acc = np.max(1 - (fpr + (1 - tpr)) / 2)
    medium = tpr[np.where(fpr < .1)[0][-1]] # TPR@10%FPR
    low = tpr[np.where(fpr < .01)[0][-1]] # TPR@1%FPR
```

七、实验设计与结果分析

Q19：论文的核心实验是如何设计的？实验矩阵是什么？

参考答案：

论文第 3 章和第 4 章共设计了**两大组实验**，形成了"3×4×4×5"的评估矩阵：

实验一（第 3 章）：攻击评估框架验证

维度	内容
数据集	CIFAR-10 (ResNet-18)、CIFAR-100 (ResNet-50)、TinyImageNet (ViT)
遗忘方法	Retrain、Finetuning (FT)、NegGrad (NG)、SCRUB
攻击方法	Unleaks、U-LiRA、RULI、ReA
遗忘比例	10% 随机样本
评估指标	Accuracy、AUC、TPR@0.1%FPR、TPR@1%FPR、TPR@10%FPR
轮次	5 轮（每轮更换 seed），报告 mean±std

实验二（第 4 章）：MUSE 防御方法验证

维度	内容
数据集	同上 3 个
遗忘方法	Retrain、FT、NG、SCRUB、 MUSE
攻击方法	同 4 种
评估指标	TA、UA、RA、MIA Efficacy（遗忘后 MIA 攻击成功率）

实验硬件：NVIDIA GeForce RTX 5090 GPU × 8

Q20：论文表 3-3（攻击实验）的核心发现是什么？

参考答案：

论文 Table 3-3 展示了 10% 随机样本遗忘下各攻击方法的 Accuracy（均值±标准差）。核心发现：

1. NegGrad（负梯度）方法最容易被攻击成功

数据集	最强攻击	Accuracy
CIFAR-10	RULI → NG	57.73% ± 0.55%
TinyImageNet	ReA → NG	63.45% ± 0.21%
CIFAR-100	RULI → NG	70.41% ± 1.65%

 NegGrad 通过在遗忘样本上做梯度上升来"反学习"，但这种粗暴操作造成模型在遗忘样本上产生异常的输出模式，反而使攻击更容易检测。

2. Retrain 是最安全的遗忘方法

- Retrain 在大多数 (数据集, 攻击) 组合下 Accuracy 接近 50%（随机猜测水平）
- 例如 CIFAR-100 + RULI → Retrain: 50.04% ± 0.65%
- 但 Retrain 的计算代价最高

3. 不同攻击方法的"擅长领域"不同

- **Unleaks**：在 CIFAR-10 上相对稳定，Accuracy 在 51.68%-54.62%

- **U-LiRA**: 跨数据集稳定性最好（方差小），对 SCRUB 较敏感
- **RULI**: 对 NG 极其敏感，在 CIFAR-100 上 Accuracy 达 70.41%
- **ReA**: 对 FT 和 NG 都较有效，TinyImageNet 上的区分度最大

4. 数据集规模与攻击难度的关系

- CIFAR-100（100 类）比 CIFAR-10（10 类）更容易被攻击（更多类别意味着更大的输出空间可被利用）
- TinyImageNet（200 类，64×64）上的攻击效果差异最大（Accuracy 从 50.41% 到 63.45%）

Q21：论文表 4-1（MUSE 对比实验）的核心发现是什么？

参考答案：

论文 Table 4-1 对比了 5 种遗忘方法在 10% 遗忘比例下的表现：

CIFAR-10 (ResNet-18) 详细数据：

方法	TA (%)	UA (%)	RA (%)	MIA Efficacy (%)
Retrain	83.24±0.35	80.51±0.58	99.99±0.01	9.29±1.40
FT	81.01±1.51	81.90±1.84	92.33±2.28	9.47±2.55
NG	72.60±0.69	50.31±0.61	80.96±0.41	10.75±2.16
SCRUB	79.81±0.88	80.61±0.51	84.11±0.59	10.15±1.16
MUSE	90.89±0.23	94.64±0.43	95.67±0.20	10.56±1.60

CIFAR-100 (ResNet-50) 详细数据：

方法	TA (%)	UA (%)	RA (%)	MIA Efficacy (%)
Retrain	69.52±0.64	67.83±0.81	98.47±0.28	8.60±1.73
FT	66.14±1.22	68.25±1.36	90.72±1.58	10.80±0.86
NG	54.36±2.07	38.74±2.41	76.53±2.35	15.93±1.89
SCRUB	61.85±1.74	62.47±1.69	82.16±1.91	8.13±2.14
MUSE	71.48±0.71	77.62±0.89	93.28±0.64	11.36±1.45

TinyImageNet (ViT) 详细数据：

方法	TA (%)	UA (%)	RA (%)	MIA Efficacy (%)
Retrain	62.35±0.72	60.48±0.91	96.82±0.36	9.40±1.06
FT	59.74±1.38	61.26±1.72	89.35±1.64	11.87±2.66
NG	47.83±2.41	32.56±2.18	73.42±2.73	20.10±1.84
SCRUB	54.67±1.96	55.31±2.04	78.85±2.26	20.61±0.68
MUSE	63.92±0.84	71.38±0.93	91.74±0.76	15.42±1.57

核心发现：

1. **MUSE 在 TA 和 UA 上全面领先：**
 - CIFAR-10: TA=90.89%（Retrain 83.24%），提升 7.65%
 - CIFAR-100: TA=71.48%（Retrain 69.52%），提升 1.96%
 - TinyImageNet: TA=63.92%（Retrain 62.35%），提升 1.57%
 - MUSE 的 TA 甚至超过 Retrain，说明分布对齐损失起到了额外的正则化作用
2. **MUSE 的 RA 接近 Retrain 水平：**
 - 在所有数据集上 RA 均在 91% 以上，说明对保留样本的影响很小

- CIFAR-10 上 RA=95.67%，仅比 Retrain 低 4.32%

3. MUSE 的 MIA Efficacy 与 FT 相当：

- 虽然高于 Retrain（理论下界），但明显优于 NG
- 在 TinyImageNet 上 MUSE (15.42%) 明显优于 SCRUB (20.61%) 和 NG (20.10%)

4. NG 是最差的方法：

- UA 极低（CIFAR-100 上仅 38.74%），说明过度损害了遗忘样本的分类性能
- MIA Efficacy 最高（CIFAR-100 上 15.93%），说明攻击者反而更容易检测

5. UA 的深层含义：

- Retrain 的 UA 反映了模型对这些样本的"天然泛化能力"（未见过但能正确分类）
- MUSE 的 UA 高于 Retrain，说明 MUSE 在消除过拟合痕迹的同时保持了良好的泛化性

Q22：论文图 4-2（ROC 曲线）揭示了什么？

参考答案：

论文 Figure 4-2 展示了在 CIFAR-10 上，不同遗忘方法面对 4 种攻击时的 ROC 曲线。

核心观察：

1. **理想情况：**完美遗忘应使 ROC 曲线与对角线重合（AUC=0.5），表示攻击者无法区分成员/非成员
2. **MUSE 的表现：**
 - MUSE 的 ROC 曲线最接近对角线
 - 面对 LiRA、ReA、Unleaks、RULI 四种攻击时，AUC 均接近 0.5
 - 说明 MUSE 成功让遗忘样本在输出上变得与非成员样本"难以区分"
3. **对比 FT/NG/SCRUB：**
 - FT 的 ROC 曲线有明显弯曲（AUC > 0.5），攻击者有一定区分能力
 - NG 的 ROC 曲线偏离对角线最远，攻击效果最好
 - SCRUB 介于 FT 和 NG 之间
4. **跨攻击方法的一致性：**
 - MUSE 对 4 种攻击都表现出较好的防御效果（通用性好）
 - 不像某些方法只对特定攻击有效

实验还证明：在 CIFAR-100 和 TinyImageNet 上，MUSE 在 TA、RA、MIA Efficacy 等指标上均表现优异，验证了方法的泛化能力。

八、工程实现与架构

Q23：项目的整体架构是如何设计的？

参考答案：

```

run_benchmark.py      ← 统一入口（一行代码转发到 benchmark/samplewise.main）
|
▼
benchmark/samplewise.py ← 核心编排器
├─ DatasetConfig      ← 数据集预设（Cifar10/Cifar100/TinyImageNet等）
├─ UnlearningMethod   ← 方法预设（含默认超参数）
|
├─ [pretrain] → pretrain_model_sample_wise.py ← 预训练目标模型
├─ [shadow]   → train.py                      ← 训练8个shadow models
├─ [unlearn]  → forget_sample_main.py         ← 执行遗忘方法
|
|          └─ forget_random_strategies.py ← 各遗忘策略实现
|              └─ finetune / retrain / negative_grad
|              └─ scrub / FisherForgetting / Wfisher
|              └─ salun / relabel / amnesiac / FT_prune
|              └─ orthogonality / sfron / rum
|              └─ muse → muse/trainer.py
├─ [reminiscence] → rea_reminiscence_random.py ← REA专用
└─ [attack]      → attacks/registry.py        ← 攻击分发
                    └─ attacks/lira.py → mia_lira.py
                    └─ attacks/rea.py  → mia_lira.py（不同task）
                    └─ attacks/ruli.py → Ruli/core/
                    └─ attacks/unlearningleaks.py → 自包含实现

```

设计模式亮点：

1. **策略模式**： forget_random_strategies.py 中的每个遗忘方法都是同一接口的函数
2. **注册模式**： attacks/registry.py 通过 @register_attack("lira") 装饰器注册攻击类
3. **适配器模式**： attacks/ 下的每个攻击类将第三方代码适配到统一的 AttackResult 接口

Q24：为什么使用 from __future__ import annotations？

参考答案：

- 这个是项目为了兼容 **Python 3.9** 而添加的（见 git history: 3ae3370 fix: add from __future__ import annotations for Python 3.9 compatibility）
- 主要作用是**延迟类型注解的求值**（PEP 563）：
 - 允许使用 list[str] 而非 List[str]（不需要从 typing 导入）
 - 支持前向引用（forward reference），即引用尚未定义的类
 - 在较旧 Python 版本中也能使用新式类型注解语法

面试扩展：

- Python 3.10+ 原生支持 x | y 联合类型语法
- Python 3.11+ 有更多类型系统改进
- 这是一个实用的工程决策：保证环境兼容性

Q25：为什么 MUSE 中要用 itertools.cycle 而不是简单的 zip？

参考答案：

代码（muse/trainer.py:76-77）：

```

forget_iter = cycle(forget_loader)
reference_iter = cycle(reference_loader)

```

原因：

- 三个 dataloader（retain、forget、reference）的长度可能不同
- 普通 zip 会在最短的 dataloader 耗尽时停止，导致数据利用不充分
- cycle 让较短的两个 loader 循环迭代，确保每个 epoch 遍历完最长的 retain_loader 的所有 batch

- 这种设计让每个 retain batch 都有对应的 forget 和 reference batch 来配对训练

九、多轮实验与可复现性

Q24：多轮实验脚本是如何保证可复现性的？

参考答案：

多轮脚本 `scripts/run_cifar10_five_rounds_muse.sh` 的设计：

1. **每轮独立的随机种子**：每轮生成一个随机 seed
2. **每轮共享的 forget sample**：同一轮内的所有方法和攻击使用相同的 forget index
3. **复用已有的 shadow checkpoints**：如果已经训练过 shadow models，直接复用
4. **失败自动跳过**：某个子实验失败不会中断整个脚本，方便长期运行
5. **自动聚合统计**：最后输出所有轮次的 mean/std

实验矩阵（5个遗忘方法 × 4个攻击方法 × 5轮 = 100个子实验）

Q25：如何向面试官展示你的实验设计能力？

参考答案：

可以从以下角度阐述：

1. **Baseline 选择的合理性**：
 - retrain 作为理论上界（但计算代价最高）
 - finetune 作为简单实用 baseline
 - negative_grad 代表对抗式方法
 - scrub 代表知识蒸馏式方法
 - muse 是你提出的新方法
2. **数据集的层次性**：
 - Cifar10（10 类，32×32，简单）→ 验证基本有效性
 - Cifar100（100 类，32×32，中等）→ 验证类别扩展性
 - TinyImageNet（200 类，64×64，较难）→ 验证更大规模场景
3. **"跨数据集 × 跨方法 × 跨攻击"三维评估**：每个方法在每个数据集上面对所有 4 种攻击的全面评估
4. **消融实验思路**：
 - $\lambda_{align} = 0$ （只用 L_{stat} ）vs $\lambda_{stat} = 0$ （只用 L_{align} ）vs 完整 MUSE
 - 不同的 λ 取值对结果的影响

十、论文研究内容与未来方向

Q26：论文明确提出的研究内容有哪几点？

参考答案：

论文 1.3 节明确了 3 项核心研究内容：

1. **面向机器遗忘的 MIA 评估框架构建**：
 - 形式化定义了样本级遗忘场景下的攻击目标
 - 构建了可适配主流实验场景的评估框架
 - 在 3 个数据集 × 3 种模型 × 4 种遗忘方法上验证
2. **多攻击方法的统一评估与分析**：
 - 对 Unleaks、LiRA、RULI、ReA 四种攻击进行系统评估
 - 从有效性、稳定性、适用性、成本、评估真实性五个维度分析

- 发现 NegGrad 最易受攻击、Retrain 最安全

3. MUSE 防御方法的提出与验证：

- 通过分布对齐和差异压缩降低 MIA 风险
- 在多个数据集上验证了方法有效性
- 提供了面向攻击防御的遗忘方法设计参考

Q27：论文提出的未来工作方向有哪些？

参考答案：

论文第 5 章提出了 5 个未来研究方向：

1. 更大规模数据集的验证：

- 扩展到 ImageNet、CINIC-10 等更大规模数据集
- 验证框架和方法在更复杂场景下的适用性

2. 与差分隐私（Differential Privacy）的结合：

- 论文参考文献 [36]（Abadi et al., 2016）提出了 DP-SGD
- DP 提供了可证明的隐私保证，但通常以牺牲模型效用为代价
- 未来可探索 MUSE 与 DP 机制的结合

3. 自适应攻击（Adaptive Attack）场景：

- 目前评估假设攻击者不知道防御方法
- 未来应评估攻击者已知 MUSE 机制时的攻击效果
- 类似对抗鲁棒性中的"自适应攻击"评估理念

4. MUSE 方法的进一步改进：

- 减少对 reference non-member dataloader 的依赖
- 自动化的超参数选择
- 扩展到 NLP 和生成模型

5. 良性数据的潜在威胁：

- 论文参考文献 [37]（Ma et al., 2024）揭示了"良性数据也可能被用于攻击"
- 未来应关注更广泛的数据安全威胁

十一、面试综合策略

参考 CS-BAOYAN 社区的保研面试体系

CS-BAOYAN（"绿群"）是全国最大的非商业计算机保研交流社区，GitHub 组织 github.com/CS-BAOYAN 提供 Wiki 知识库、DDL 时间线追踪、经验贴汇总、复习资料等核心资源。以下内容融合了 CS-BAOYAN 社区的面试准备体系与本项目的实际技术内容。

"各位老师好，我是 XXX，来自 XX 大学 XX 专业。在本科期间，我深入参与了一个关于 **机器遗忘与隐私保护** 的科研项目——MIA Benchmark + MUSE。

这个项目构建了一个**统一的样本级机器遗忘基准测试平台**，接入 4 种主流的成员推理攻击方法（LiRA、REA、RULI、UnlearningLeaks）和 5 种以上的遗忘方法。在此基础上，我提出了一种新的遗忘方法 **MUSE**，核心思路是让被遗忘样本在模型输出统计特征上与从未训练过的样本对齐，从而降低被攻击者识别为"成员"的风险。

项目使用 **PyTorch** 实现，支持 Cifar10/Cifar100/TinyImageNet 等多数据集和 ResNet/ViT 等多骨干网络，设计了统一的多轮实验框架确保结果的可复现性。我主要负责了 [你具体负责的部分，如：MUSE 方法的算法设计、多攻击方法的统一接入、benchmark 框架的架构设计、实验评估与分析]。

我对 AI 安全与隐私保护方向非常感兴趣，希望能在这个领域继续深入研究。"

二、常见追问及应对

追问 1："你做这个项目的 motivation 是什么？"

回答思路：

1. 法规驱动（GDPR、中国个人信息保护法）

- 2. 现有研究的不足：缺乏统一的 benchmark、各方法评估条件不一致
- 3. 对"如何在遗忘效果和模型效用之间取得平衡"这一核心问题的兴趣

追问 2： "你觉得你提出的 MUSE 方法最大的局限性是什么？ "

回答思路：

- 1. 需要额外的 reference non-member dataloader（实际场景中可能不容易获取足够的非成员数据）
- 2. batch-mean 对齐可能对 outlier 样本效果不佳
- 3. 超参数对不同数据集的敏感度较高，需要经验性调参
- 4. 展示批判性思维比回答内容本身更重要

追问 3： "如果让你改进这个项目，你会怎么做？ "

回答思路：

- 1. 支持更多模型类型：目前主要支持 ResNet 和 ViT，可以扩展到 LLM 的遗忘评估
- 2. 自适应攻击：增加 adaptive attack 场景，让攻击者知道遗忘方法并针对性绕过
- 3. 理论分析：给出 MUSE 方法的理论保证（如差分隐私的 ϵ 界）
- 4. 自动化超参数搜索：减少人工调参的工作量

追问 4： "你怎么判断一个遗忘方法是否真的'安全'？ "

回答思路：

- 没有任何遗忘方法能做到"绝对安全"
- 安全是相对的，取决于攻击者的能力和评估的严格程度
- 应该从多维度评估：MIA AUC/TPR、遗忘集准确率 UA、模型效用 TA/RA
- 长远来看，差分隐私（Differential Privacy）提供了可证明的安全性保证

三、面试常见基础问题速查表（CS-BAOYAN 高频考点）

这些是计算机保研面试（参考 CS-BAOYAN 社区高频题库）中与你的项目直接关联的基础知识点：

知识点	与项目的关联	CS-BAOYAN 高频题型
交叉熵损失 (CrossEntropyLoss)	MUSE 的 L_base、所有分类训练的基础损失函数	损失函数对比、多分类问题
KL 散度 / JS 散度	ZRF 指标的核心计算、SCRUB 方法的蒸馏损失	信息论基础、概率分布度量
Softmax 函数	模型输出的概率化、MUSE 的分布对齐目标	激活函数对比、梯度消失
SGD vs Adam 优化器	ResNet 用 SGD、ViT 用 AdamW	优化器选择依据、momentum 原理
学习率调度 (MultiStepLR)	train.py 中的 milestone 机制	过拟合/欠拟合判断
过拟合与正则化	L2 正则化、early stopping	偏差-方差权衡
ROC 曲线与 AUC	MIA 攻击效果的核心评估指标	混淆矩阵、PR 曲线
高斯分布 / 似然比检验	LiRA 攻击的数学基础	概率论基础、假设检验
卷积神经网络 (CNN)	ResNet-18/ResNet-50 骨干网络	卷积/池化/全连接原理
Vision Transformer (ViT)	TinyImageNet 骨干网络	Self-Attention 机制、位置编码
数据增强 (Data Augmentation)	LiRA 中生成 canary 变体	常用增强策略
随机种子与可复现性	所有实验的 set_random_seed	实验设计原则

三-B、CS-BAOYAN 高频专业课考点补充

面试中除了项目本身，面试官可能追问以下专业课问题以考察你的基础功底：

数据结构与算法（优先级 ★★★★★）：

- 快速排序的思想与时间复杂度？（你的项目中 np.random.choice 底层的随机采样）

- 哈希表的冲突解决方法？（项目中的 forget index 映射）
- BFS 与 DFS 的区别？（可用于理解模型搜索空间）

操作系统（优先级 ★★★★★）：

- 进程与线程的区别？（PyTorch DataLoader 的 num_workers 参数）
- 虚拟内存的原理？（GPU 显存管理、大规模数据加载）

计算机网络（优先级 ★★★★★）：

- TCP 三次握手？（分布式训练中的通信协议基础）
- HTTP vs HTTPS？（API 安全、数据传输加密）

概率论与数理统计（优先级 ★★★★★）：

- 贝叶斯公式？（MIA 攻击的数学基础，lira_online 中的似然比）
- 高斯分布的参数估计？（LiRA 的核心建模）
- 什么是 p-value？（假设检验与 MIA 决策）

线性代数（优先级 ★★★）：

- 特征值与特征向量的含义？（PCA 降维、模型参数矩阵分析）
- 矩阵的秩？（模型表达能力分析）

四、应答黄金法则（融合 CS-BAOYAN 社区经验）

1. **STAR 法则**（描述项目经历）：

- **Situation**：机器遗忘是 AI 隐私合规的关键技术（引 GDPR、"被遗忘权"）
- **Task**：现有评估体系碎片化，缺乏统一 benchmark 和有效的防御方法
- **Action**：① 构建统一评估框架（3 数据集 × 4 方法 × 4 攻击 × 5 维度）；② 提出 MUSE 防御方法
- **Result**：系统评估了 48 组实验组合，MUSE 在保持模型效用的同时降低了隐私泄露风险

2. **"是什么 → 为什么 → 怎么做"** 三段论：

- 是什么：机器遗忘 + 成员推理攻击
- 为什么：隐私法规（GDPR/个人信息保护法）、现有评估不充分
- 怎么做：统一框架 + MUSE（分布对齐 + 统计量匹配）

3. **主动引导**：在回答中自然引出你擅长的技术点

- 例如："在实现过程中，我对 MUSE 的损失函数设计特别感兴趣，因为论文表 4-1 显示它在 CIFAR-10 上将 TA 从 83.24% 提升到 90.89%....."

4. **承认边界**：

- "这个我目前了解不深，但根据我对 MIA 的理解....."
- 展示诚实和学习能力 > 假装全知

5. **双向选择意识**（CS-BAOYAN 核心理念）：

- 面试是双向选择，不是"被审问"
- 适当反问："老师您觉得机器遗忘在实际部署中最大的挑战是什么？"
- 展示独立思考和学术兴趣

6. **导师方向匹配**（CS-BAOYAN 核心策略）：

- 提前了解目标导师的研究方向（CV/NLP/安全/系统）
- 如果导师做 AI 安全：强调项目中 MIA 评估和 MUSE 防御
- 如果导师做 CV：强调 ResNet/ViT 模型的训练和评估经验
- 如果导师做系统工程：强调 benchmark 框架的工程设计和可复现性

五、英语面试准备（CS-BAOYAN 常见题型）

1 分钟英文自我介绍模板：

"Good morning professors. I'm [Name] from [University], majoring in [Major]. During my undergraduate studies, I participated in a research project on **machine unlearning and privacy protection**. We built a unified benchmark framework for evaluating membership inference attacks in machine unlearning scenarios, and I proposed a defense method called MUSE — Membership-Undistinguishable Sample Erasure. The core idea is to align the output distribution of forgotten samples with that of non-member reference samples, making them indistinguishable to attackers. We validated our approach on CIFAR-10, CIFAR-100 and TinyImageNet across multiple model architectures. I'm particularly interested in AI security and privacy, and I hope to continue research in this direction."

常见英语追问（可提前准备）：

问题	准备要点
Why this university?	提及目标导师的研究方向，展示诚意
What's your favorite course?	选与项目相关的课（机器学习、概率统计）
What are your strengths/weaknesses?	用项目经历佐证
Describe your project in 2 minutes	用英文三句话讲清楚：Problem → Method → Result
What's the difference between MIA and differential privacy?	MIA 是攻击手段，DP 是防御框架，DP 提供可证明保证

六、CS-BAOYAN 资源速查

资源	说明	链接/渠道
CS-BAOYAN-Wiki	历年夏令营/预推免时间、院校解析、经验贴	github.com/CS-BAOYAN/CS-BAOYAN-Wiki
CS-BAOYAN-DDL	各院校报名截止时间实时跟踪	cs-baoyan.github.io/cs-baoyan-ddl/
CSBasicKnowledge	计算机/AI/EE 基础知识整理	github.com/CS-BAOYAN/CSBasicKnowledge
绿群 QQ 群	实时交流（大三同学）	943826679 / 620876705
经验贴汇总	往届学生完整保研经历	CS-BAOYAN-Wiki 中分类整理
面试回忆	各院校面试真题回忆	GitHub Discussions / QQ 群文件

附录：项目关键技术速查

A. 关键文件索引

文件	功能
run_benchmark.py	统一入口
benchmark/samplewise.py	核心编排器（675行）
forget_sample_main.py	遗忘主入口
forget_random_strategies.py	所有遗忘策略实现（1476行）
muse/trainer.py	MUSE 核心训练逻辑（166行）
mia_lira.py	LiRA/REA 攻击执行（298行）
lira_utils.py	LiRA 工具函数（767行）
unlearn.py	基础遗忘算法模块（212行）
train.py	目标模型和 Shadow 模型训练（309行）
config.py	全局配置（104行）
attacks/base.py	攻击上下文和结果的数据类
attacks/lira.py	LiRA 攻击适配器
attacks/rea.py	REA 攻击适配器
attacks/ruli.py	RULI 攻击适配器（343行）
attacks/unlearningleaks.py	UnlearningLeaks 攻击适配器（552行）
utils/metrics.py	ZRF、MIA 指标实现

B. 核心类与函数索引

类/函数	位置	功能
AttackContext	attacks/base.py:11	攻击上下文数据类
AttackResult	attacks/base.py:57	攻击结果统一封装
MUSEConfig	muse/trainer.py:13	MUSE 配置数据类
train_muse_epoch	muse/trainer.py:59	MUSE 单轮训练
compute_alignment_loss	muse/trainer.py:27	分布对齐损失
compute_stat_loss	muse/trainer.py:44	统计量对齐损失
UnLearningScore	utils/metrics.py:20	ZRF 计算
get_membership_attack_prob	utils/metrics.py:74	MIA 概率计算
lira_online	lira_utils.py:320	LiRA 在线攻击
lira_offline	lira_utils.py:361	LiRA 离线攻击
cal_results	lira_utils.py:403	LiRA 结果汇总
evaluate	utils/overall_utils.py	模型评估
fit_one_cycle	utils/overall_utils.py	训练一个完整周期

C. 命令行速查

```
# 最简示例
python run_benchmark.py --dataset Cifar10 --seed 1 --methods finetune --attacks rea

# 完整流程
python run_benchmark.py --dataset Cifar10 --seed 1 \
  --methods finetune --attacks lira rea ruli unlearningleaks \
  --stages pretrain shadow unlearn reminiscence attack

# 干运行检查
python run_benchmark.py --dataset Cifar10 --seed 1 \
  --methods finetune --attacks lira rea --stages unlearn attack --dry-run

# MUSE 自定义参数
python run_benchmark.py --dataset Cifar10 --seed 1 \
  --methods muse:4e-4:7 --attacks rea --stages unlearn reminiscence attack

# 多轮实验
PYTHON_BIN=/path/to/python bash ./scripts/run_cifar10_five_rounds_muse.sh
```

结语：保研面试的核心在于“让面试官相信你真正理解了你所做的项目”。本文档涵盖了论文与代码从概念到实现的各个层面，建议结合论文原文和代码实际运行一遍，深入理解每个环节的技术细节。面试时不求面面俱到，但求对核心问题的深入思考和有逻辑的表达。祝复试顺利！

附录 D：论文核心参考文献速查

编号	作者/会议	核心贡献	与本项目的关联
[4]	Shokri et al., IEEE S&P 2017	MIA 奠基性工作	MIA 基本概念来源
[6]	Carlini et al., IEEE S&P 2022	LIRA 攻击	本项目 LiRA 攻击的实现基础

编号	作者/会议	核心贡献	与本项目的关联
[9]	Bourtoule et al., IEEE S&P 2021	机器遗忘概念提出	本项目遗忘方法的分类基础
[15]	Chen et al., ACM CCS 2021	机器遗忘中的隐私泄露	Unleaks 攻击方法来源
[17]	Kurmanji et al., NeurIPS 2023	SCRUB 遗忘方法	本项目 SCRUB baseline
[19]	Xiao et al., ICCV 2025	ReA 攻击	本项目 reminiscence 阶段来源
[20]	Gupta et al., NeurIPS 2021	自适应机器遗忘 (AMU)	攻击评估参考
[21]	Naderloui et al., USENIX 2025	RULI 攻击	本项目 RULI 适配来源
[30]	Graves et al., AAAI 2021	Amnesiac 遗忘方法	随机标签遗忘策略
[31]	Thudi et al., EuroS&P 2022	NegGrad 遗忘方法	本项目负梯度策略来源
[34]	Tang et al., arXiv 2025	Apollo 攻击	攻击方法参考
[35]	Li et al., USENIX 2025	DHAttack	攻击方法参考
[36]	Fan et al., ICLR 2024	SalUn 遗忘方法	基于权重显著性的遗忘
[37]	Ma et al., arXiv 2024	良性数据威胁	论文未来工作方向 5 的来源